

Summary and Reflections Report: Software Testing for Back-End Services

Matthew Wood

Computer Science Department, Southern New Hampshire University

CS 320: Software Test Automation

Professor Rob Tuft

February 19, 2026

Summary

Alignment to Requirements

My unit testing approach for the contact, task, and appointment features for Grand Strand Systems was fundamentally driven by the software requirements provided in Project One. Before writing any code, I analyzed the constraints for each service to ensure that the "before" thought process was aligned with the customer's needs. For example, the requirements stated that a Contact ID must be a non-null string of exactly 10 characters. I aligned my testing to these rules by creating boundary cases that specifically targeted these limits. In the Task service, I included tests to verify that a description exceeding 50 characters would trigger an `IllegalArgumentException`, which directly validated the requirement for data integrity. By testing the edges of the requirements, I ensured that the back-end services would reject invalid data before it could cause systemic issues.

Defending Quality and Effectiveness

The quality of my JUnit tests is evidenced by the 100% code coverage achieved within the Eclipse environment. However, coverage is only effective if the tests themselves are meaningful. I ensured effectiveness by testing "negative" paths—the scenarios where the code should fail. By verifying that every if statement responsible for error handling was executed and passed, I confirmed that the services were robust enough to handle improper inputs without crashing. This comprehensive coverage ensures that no hidden logical branches exist that could lead to unpredictable behavior in a production environment.

Technically Sound and Efficient Code

To ensure the code was technically sound, I utilized a "test-first" mentality where I considered potential failure points before finalizing the service logic. I used specific JUnit 5 features like `assertThrows` to confirm that my constructors properly rejected invalid data. For example:

```
assertThrows(IllegalArgumentException.class, () -> { new Contact(null, "John", "Doe",  
"1234567890", "Address"); });
```

To maintain efficiency, I leveraged the Maven structure and the `@BeforeEach` annotation. This allowed me to reset the environment before every test, preventing redundant object creation and ensuring that a failure in one test did not skew the results of another. For instance, by initializing a fresh `TaskService` in a setup method, I kept the test suite fast and maintainable.

Reflection

Testing Techniques Employed

In this project, I primarily employed **Unit Testing** and **White-Box Testing**. Unit testing allowed me to isolate the logic of individual classes like `Appointment` to ensure they functioned correctly in a vacuum. White-box testing was essential because it allowed me to look at the internal structure of my Java classes to ensure that every if-else condition was covered by at least one test case. I also utilized **Black-Box Testing**, where I ignored the internal code and tested purely against the requirements—checking if the output of a method matched the customer's specifications regardless of how the code reached that result.

Other Software Testing Techniques

Beyond the unit-level testing performed here, several other techniques are vital for full-scale development at Grand Strand Systems. These techniques are recognized as critical components of the international test process standards:

- **Integration Testing:** This involves testing how multiple modules work together. While I verified the Contact class worked, an integration test would verify if the ContactService correctly stores those objects in a database.
- **Regression Testing:** This is performed after a code change to ensure that new features haven't broken existing functionality. In a continuous testing environment, this is often automated via Maven.
- **System Testing (UAT):** User Acceptance Testing involves the actual client testing the software to ensure it meets their real-world business needs.

Practical Uses and Implications

The practical use of unit testing is to catch logic errors at the "atomic" level where they are cheapest to fix. However, the implication of only using unit testing is that "handshake" errors—where two working parts fail to communicate—can go unnoticed until late in the development cycle. Integration testing is necessary to prevent these communication failures, while regression testing is the only way to ensure that as a project grows, it doesn't collapse under its own weight. My documentation of these results functions as a "Test Completion Report," a professional artifact described in the **ISO/IEC/IEEE 29119-3:2021** standard to summarize testing activities and identify remaining risks for stakeholders.

Mindset

Caution and Complexity

Acting as a software tester required a high degree of caution. My "before" thought process involved recognizing that even "simple" back-end services have complex interrelationships. I had to appreciate this complexity because a single error in the Appointment date validation could result in a system-wide scheduling failure. This focus on "negative paths" mirrors the risk-based approach recommended by modern international standards to ensure that high-priority system components are rigorously verified.

Limiting Bias

As the developer of Project One, I recognized an inherent "Happy Path" bias—the tendency to only test scenarios where everything works perfectly. To limit this, I adopted a destructive testing mindset "after" the development phase. I purposefully tried to break my own code by passing null values and empty strings into the Task service. This shift in perspective was essential to ensure that the software was resilient against real-world errors.

Discipline and Technical Debt

Finally, I evaluated the importance of being disciplined in my commitment to quality. Cutting corners by skipping a unit test is the fastest way to accumulate technical debt. This debt is a burden that the team eventually has to pay back with interest through emergency bug fixes. I plan to avoid technical debt by making testing a continuous part of my Maven-based loop; if a feature isn't tested, it isn't finished. This disciplined documentation of data requirements and execution results ensures that the code remains maintainable and auditable.

<div style="page-break-after: always;"></div>

References

Aniche, M. (2022). *Effective software testing: A developer's guide*. Manning Publications.

<https://learning.oreilly.com/library/view/effective-software-testing/9781633439931/>

International Organization for Standardization. (2021). *Software and systems engineering—Software testing—Part 3: Test documentation* (ISO/IEC/IEEE Standard No. 29119-3:2021).

<https://www.iso.org/standard/79429.html>